National University of Singapore
School of Computing
CS1101S: Programming Methodology
Semester I, 2026/2027

R13
Typed Python

Python §4 Typed

Python §4 Typed is Python §4 with *type annotations*. Choose it as the language in the Source Academy Playground. A program is first type-checked, and runs only if the checker finds no type errors.

`int`, `float`, `bool`, `str`, `None`	basic types; an `int` can be used where a `float` is expected
`Callable[[A, B], C]`	functions taking an `A` and a `B`, returning a `C`
`Pair[H, T]`	pairs whose head has type `H` and whose tail has type `T`
`LList[T]`	linked lists (predeclared, below)
`list[T]`	the mutable lists of Python §4
`T | None`	a `T`, or `None`
`def f[A](x: A) -> A:`	a generic function: `A` stands for any type

```
# predeclared
type LList[T] = Pair[T, LList[T]] | None
```

Every parameter and return type must be annotated. Conditions must be of type `bool`, and a `bool` cannot be used as a number. After a test `xs is None` (or `is_none(xs)`), the checker knows in each branch whether `xs` is `None`.

Problems:

1. Each program below has a type error. For each one, first *predict* what the checker will complain about, and where. Then run it, and fix the program.

   (a)

   ```
   def label(n: int) -> str:
       return "item " + n
   ```

(b)

```
def is_empty(xs: LList[int]) -> bool:
    if xs:
        return False
    else:
        return True
```

(c)

```
def second(xs: LList[int]) -> int:
    return head(tail(xs))
```

2. Here are functions you wrote in Python §2, without types. Add type annotations until the checker accepts all of them. They work for linked lists of any element type, so they need to be generic, like `def f[A](x: A) -> A:`.

```
def length(xs):
    return 0 if is_none(xs) else 1 + length(tail(xs))

def map(f, xs):
    return (None
            if is_none(xs)
            else pair(f(head(xs)), map(f, tail(xs))))

def filter(pred, xs):
    return (None
            if is_none(xs)
            else pair(head(xs), filter(pred, tail(xs)))
                if pred(head(xs))
                else filter(pred, tail(xs)))

def take(xs, n):
    return None if n == 0 else pair(head(xs), take(tail(xs), n - 1))

xs = llist(1, 2, 3)
print(length(xs))
print_llist(map(lambda x: 2 * x, xs))
print_llist(filter(lambda x: x % 2 == 1, xs))
print_llist(take(xs, 2))
```

Your annotated `take` will still be rejected. Why? Fix it.

3. (a) Write a function `find` with type `(LList[int], int) -> int | None` that returns the position of the first occurrence of a given number in a linked list (counting from 0), or `None` if it does not occur.

   (b) Write a function `position_after` with type `(LList[int], int) -> int` that returns the position after the first occurrence, using `find`, or `-1` if the number does not occur. Why does the checker reject `return find(xs, target) + 1`?